

T.C.
İSTANBUL NİŞANTAŞI ÜNİVERSİTESİ
LİSANSÜSTÜ EĞİTİM ENSTİTÜSÜ
BİLGİSAYAR MÜHENDİSLİĞİ TEZSİZ YÜKSEK LİSANS PROGRAMI

BİTİRME PROJESİ RAPORU

ColdStart: Seyrek Bilgi Ortamları için Adaptif Hibrit RAG
Mimarisi

Hazırlayan

Kubilay ÖZYALÇIN

Bilgisayar Mühendisliği Tezsiz Yüksek Lisans

20251575002

Danışman

Prof. Dr. Yaşar HOŞCAN

İstanbul, Haziran 2026

Özet

Retrieval-Augmented Generation (RAG) sistemleri, büyük dil modellerinin cevaplarını dış bir bilgi tabanından getirilen belgelerle temellendirerek uydurma (halüsinasyon) riskini azaltır. Ancak bu sistemlerin literatürdeki tasarımı, elde yeterince belge olduğu varsayımına dayanır. Sistem ilk kurulduğunda bilgi tabanı boştur; embedding üretilmemiştir ve anlamsal arama yapılamaz. Bu çalışmada, cold start adı verilen bu probleme yönelik üç katmanlı ve adaptif bir RAG mimarisi tasarladım ve .NET 8 platformunda uçtan uca çalışan bir prototip olarak geliştirdim.

Önerdiğim mimaride sistem, kayıtlı belge sayısını her istekte okuyarak uygun arama stratejisini çalışma anında seçer. Belge sayısı azken BM25 tabanlı anahtar kelime araması (Layer 1), orta seviyede OpenAI embedding ve kosinüs benzerliğine dayalı anlamsal arama (Layer 2), yeterli belge birikince Qdrant vektör veri tabanı ve GPT-4o-mini ile tam RAG akışı (Layer 3) devreye girer. Katmanlar arası geçiş iki yönlü ve otomatiktir.

Sistemin kalitesini LLM-as-a-judge yöntemiyle ölçtüm. Deneylerde katman geçiş doğruluğu 30 ölçümün tamamında yüzde yüz çıktı. Küçük corpus üzerinde adaptif yaklaşım, yalnız embedding kullanan baseline'dan daha alakalı sonuçları yaklaşık bin kat daha kısa sürede ve sıfır LLM maliyetiyle verdi. Layer 1 ile Layer 2'nin kalite eğrileri yaklaşık 50 belge civarında kesişti; bu bulgu, mimaride kullandığım geçiş eşiğinin ampirik dayanağını oluşturdu.

Anahtar Kelimeler: Retrieval-Augmented Generation, cold start, hibrit arama, BM25, embedding, vektör veri tabanı, LLM-as-a-judge, .NET

Abstract

ColdStart: An Adaptive Hybrid RAG Architecture for Sparse Knowledge Environments

Retrieval-Augmented Generation (RAG) systems ground the answers of large language models in documents retrieved from an external knowledge base, reducing the risk of hallucination. However, their design in the literature assumes a sufficiently populated knowledge base. When a system is first deployed, the knowledge base is empty; no embeddings exist and semantic search is impossible. In this work, I designed a three-layer adaptive RAG architecture targeting this cold start problem and implemented it as an end-to-end prototype on .NET 8.

In the proposed architecture, the system reads the current document count on every request and selects the appropriate retrieval strategy at runtime: BM25 keyword search (Layer 1) for sparse collections, OpenAI embeddings with cosine similarity (Layer 2) for medium density, and a full RAG pipeline over the Qdrant vector database with GPT-4o-mini (Layer 3) once enough documents accumulate. Transitions between layers are bidirectional and automatic.

Quality was measured with the LLM-as-a-judge method. Layer transition accuracy was 100% across 30 measurements. On small corpora the adaptive approach returned more relevant results than an embedding-only baseline, roughly a thousand times faster and at zero LLM cost. The quality curves of Layer 1 and Layer 2 intersected at around 50 documents, providing empirical support for the transition threshold used in the architecture.

Keywords: Retrieval-Augmented Generation, cold start, hybrid retrieval, BM25, embeddings, vector database, LLM-as-a-judge, .NET

İçindekiler

Özet.....	2
Abstract.....	3
İçindekiler.....	4
1. Giriş.....	6
1.1 Problem Tanımı.....	6
1.2 Amaç ve Kapsam.....	6
1.3 Raporun Yapısı.....	7
2. Literatür Taraması.....	7
2.1 Retrieval-Augmented Generation.....	7
2.2 Hibrit Arama Yaklaşımları.....	7
2.3 Cold Start Problemi.....	8
2.4 RAG Değerlendirme Yöntemleri.....	8
3. Sistem Mimarisi ve Yöntem.....	8
3.1 Genel Bakış.....	8
3.2 Layer 1: Anahtar Kelime Araması.....	9
3.3 Layer 2: Hafif Anlamsal Arama.....	9
3.4 Layer 3: Tam RAG.....	9
3.5 Yazılım Mimarisi Kararları.....	10
4. Deneysel Çalışma ve Bulgular.....	11
4.1 Değerlendirme Yöntemi.....	11
4.2 Katman Geçiş Doğruluğu.....	11
4.3 Strateji Karşılaştırması.....	11
4.4 Eşik Hassasiyeti.....	12

4.5 Katman Kalite Asimetrisi.....	12
5. Sonuç ve Gelecek Çalışmalar.....	13
5.1 Sonuç.....	13
5.2 Gelecek Çalışmalar.....	13
Kaynakça.....	14

1. Giriş

1.1 Problem Tanımı

RAG, bir dil modelinin cevap üretmeden önce dış bir bilgi tabanından soruyla ilgili belgeleri çekmesi ve cevabını bu belgelere dayandırması tekniğidir [1]. Yöntem, modelin kendi parametrik bilgisine güvenmek yerine kaynak gösterilebilir cevaplar üretmesini sağlar. Son yıllarda kurumsal soru cevap sistemlerinin fiilen standardı haline gelmiştir [2].

Bu mimarinin gözden kaçan bir ön koşulu var: çalışması için dolu bir bilgi tabanı gerekir. Sistem ilk kurulduğunda ortada belge yoktur. Embedding üretilmemiştir, vektör veri tabanı boştur, anlamsal arama yapacak bir şey yoktur. Bu duruma cold start problemi diyorum. Literatürde bu kavram daha çok öneri sistemleri bağlamında incelenmiş [5]; bilgi araması yapan RAG sistemleri için sistematik bir çözüm önerisine taramalarımnda rastlamadım.

Problemin pratik tarafı da önemli. Az belgeyle embedding tabanlı arama yapmak hem zayıf sinyal üretir hem de her sorguda API maliyeti doğurur. Üstüne LLM ile cevap üretirmek, dayanak azken uydurma riskini büyütür. Yani sorun yalnızca sistemin çalışmaması değil; az veride yanlış stratejiyle çalışmasıdır.

1.2 Amaç ve Kapsam

Bu projedeki amacım, bilgi tabanı boşken bile anlamlı cevap verebilen ve veri biriktikçe arama stratejisini kendisi değiştiren bir soru cevap sistemi geliştirmektir. Bunun için üç katmanlı bir mimari tasarladım: belge sayısı azken klasik anahtar kelime araması, orta seviyede anlamsal arama, yeterli belge birikince tam RAG. Katman seçimini kullanıcı değil sistem yapar; karar her istekte belge sayısına bakılarak verilir.

Kapsam üç fazda ilerledi. İlk iki fazda Layer 1 ve Layer 2 ile çalışan demo, bellek içi belge deposu ve HTTP API geliştirdim. Üçüncü fazda Layer 3'ü (Qdrant, Semantic Kernel, GPT-4o-mini) tamamladım, ölçüm altyapısını kurdum ve deneyleri koştum. Kalıcı depolama, kimlik doğrulama gibi üretim konularını bilinçli olarak kapsam dışında bıraktım; bunlar gelecek çalışma bölümünde ele alınıyor.

1.3 Raporun Yapısı

İkinci bölümde literatürü özetliyorum ve bu çalışmanın boşluğun neresine oturduğunu anlatıyorum. Üçüncü bölüm sistem mimarisini, katmanları ve aldığım tasarım kararlarını içeriyor. Dördüncü bölümde değerlendirme yöntemini ve deney sonuçlarını veriyorum. Beşinci bölüm sonuç ve gelecek çalışmalara ayrıldı.

2. Literatür Taraması

2.1 Retrieval-Augmented Generation

RAG kavramını sistematik biçimde ortaya koyan çalışma Lewis ve arkadaşlarına ait [1]. Yazarlar, bilgi yoğun görevlerde üretken modeli yoğun vektör aramasıyla birleştirerek hem doğruluğu artırmış hem de modelin bilgi kaynağını güncellenebilir hale getirmişlerdir. Gao ve arkadaşlarının kapsamlı taraması [2], bu temel fikrin etrafında oluşan teknikleri Naive RAG, Advanced RAG ve Modular RAG başlıkları altında toplar. Taramada dikkatimi çeken nokta şuydu: incelenen tüm varyantlar, retrieval yapılacak corpus'un hazır olduğunu varsayıyor.

Asai ve arkadaşlarının Self-RAG çalışması [3], modelin ne zaman retrieval yapacağına ve getirilen belgelerin işe yarayıp yaramadığına kendi kendine karar vermesini öğretir. Bu çalışma retrieval kararını dinamikleştirme bakımından projeme fikir verdi; ancak Self-RAG'in dinamizmi sorgu düzeyindedir, corpus'un büyüklüğüne göre strateji değiştirmez.

2.2 Hibrit Arama Yaklaşımları

Seyrek (sparse) ve yoğun (dense) arama yöntemlerini birleştiren hibrit yaklaşımlar literatürde iyi çalışılmış durumda [4]. Tipik kurulum, BM25 ile vektör aramasını aynı anda çalıştırıp sonuçları birleştirmektir. Benim yaklaşımım bundan ayrılıyor: yöntemleri paralel değil sıralı kullanıyorum. Hangi yöntemin aktif olacağını veri yoğunluğu belirliyor. Bu sayede az veride embedding ve LLM maliyeti hiç oluşmuyor.

BM25'in kendisi Robertson ve Zaragoza'nın olasılıksal alaka çerçevesine dayanır [6]. Terim sıklığı ile ters belge sıklığını birleştiren bu fonksiyon, özellikle küçük corpus'larda kelime düzeyinde yüksek isabet verir. Layer 1'i BM25 üzerine kurma kararında bu özellik belirleyici oldu: algoritma deterministiktir, dış servise bağımlı değildir ve tek belgeyle bile çalışır.

2.3 Cold Start Problemi

Cold start kavramı öneri sistemleri literatüründe uzun süredir biliniyor: sisteme yeni katılan kullanıcı veya ürün hakkında veri yoktur ve öneri yapılamaz. Chen ve Xu'nun ColdRAG çalışması [5], RAG tekniklerini bu probleme uygular; ancak hedefi öneri kalitesini artırmaktır. Bilgi tabanlı soru cevap sistemlerinde corpus'un kendisinin boş olması durumu, bildiğim kadarıyla, sistematik olarak ele alınmamıştır. Bu projenin literatüre katkısı tam bu noktada: cold start'ı retrieval stratejisi seçimi problemi olarak tanımlamak ve veri yoğunluğuna duyarlı katmanlı bir çözüm önermek.

2.4 RAG Değerlendirme Yöntemleri

RAG sistemlerinin değerlendirmesinde son dönemin standardı, cevap kalitesini başka bir dil modeline puanlatmaktır. RAGAS çerçevesi [7] faithfulness (cevabın kaynaklara sadakati) ve answer relevancy (cevabın soruyla alakası) metriklerini tanımlar. LLM-as-a-judge yaklaşımının insan değerlendirmesiyle yüksek uyum gösterdiği Zheng ve arkadaşlarının çalışmasında raporlanmıştır [8]. Ben de deneylerimde bu iki metriği, GPT-4o-mini'yi hakem olarak kullanarak ölçtüm.

Güvenlik tarafında Greshake ve arkadaşları [9], retrieval ile gelen belge içeriğine gizlenmiş talimatların LLM'i yanıltabildiğini göstermiştir (indirect prompt injection). Layer 3'ü tasarlarırken bu saldırı sınıfına karşı spotlighting tekniğini uyguladım; ayrıntısı mimari bölümünde.

3. Sistem Mimarisi ve Yöntem

3.1 Genel Bakış

ColdStart, ASP.NET Core 8 üzerinde çalışan bir web servisi ve ona eşlik eden tek sayfalık bir demo arayüzünden oluşuyor. Sistemin kalbi PipelineRouter sınıfı: her arama isteğinde belge deposundaki kayıt sayısını okuyor ve isteği üç katmandan uygununa yönlendiriyor.

Katmanların üçü de aynı ISearchLayer arayüzünü uyguladığı için router hiçbirinin iç detayını bilmiyor.

Katman	Aralık	Teknoloji	LLM
Layer 1: Keyword	belge < 50	BM25 (k1=1.5,	Yok

		b=0.75), Türkçe duyarlı tokenizer	
Layer 2: Lightweight	$50 \leq \text{belge} < 200$	OpenAI text-embedding-3-small + kosinüs benzerliği	Yok
Layer 3: Full RAG	$\text{belge} \geq 200$	Qdrant + Semantic Kernel + GPT-4o-mini	Var

Eşik değerleri koda gömülü değil; appsettings.json dosyasından okunuyor. Demo modunda 5 ve 20, üretim modunda 50 ve 200 kullanıyorum. 50 değerinin deneysel gerekçesini dördüncü bölümde gösteriyorum. Geçiş iki yönlü: belgeler silinirse sistem üst katmandan alta kendiliğinden iniyor.

3.2 Layer 1: Anahtar Kelime Araması

İlk katman BM25 algoritmasını [6] kullanıyor. Belgeler küçük harfe indirilip Türkçe karakterlere duyarlı biçimde tokenize ediliyor; skorlama TF-IDF mantığının doygunluk düzeltilmeli halidir. Bu katmanın değeri sadeliğinde: dış servis yok, ağ gecikmesi yok, maliyet yok. Sistem tek belgeyle hatta sıfır belgeyle bile anlamlı bir cevap (en azından durumu açıklayan bir mesaj) dönebiliyor. Cold start iddiasının teknik temeli bu katman.

3.3 Layer 2: Hafif Anlamsal Arama

İkinci katman, sorguyu ve belgeleri OpenAI'nın text-embedding-3-small modeliyle 1536 boyutlu vektörlere çevirip kosinüs benzerliğiyle sıralıyor. Burada bilinçli bir tasarım kararım var: embedding'leri belge eklenirken değil, ihtiyaç anında üretiyorum (lazy yaklaşım). Eksik embedding'ler ilk Layer 2 sorgusunda toplu istekle hesaplanıp belgeye geri yazılıyor. Corpus Layer 1 bölgesindeyken hiçbir embedding maliyeti oluşmuyor. Bu katman LLM ile cevap üretmiyor; en alakalı belgelerin özetlerini kaynak skorlarıyla birlikte döndürüyor.

3.4 Layer 3: Tam RAG

Üçüncü katman klasik RAG akışını uyguluyor: retrieve, augment, generate. Belgeler 800 karakterlik, 100 karakter örtüşmeli parçalara (chunk) bölünüyor; kelime ortasından kesmemek için pencere sınırları boşluğa hizalanıyor. Parçalar embedding'leriyle birlikte Qdrant vektör

veri tabanına yazılıyor. Sorgu geldiğinde en yakın parçalar bulunuyor ve GPT-4o-mini'ye bağlam olarak veriliyor; model cevabı bu bağlamdan üretiyor ve dayandığı belge kimliklerini köşeli parantezle belirtiyor.

Qdrant indeksi ile belge deposunun tutarlılığını lazy senkronizasyonla sağlıyorum. Her Layer 3 aramasının başında indeksteki belge listesi depoya karşılaştırılıyor: eksik belgeler parçalanıp ekleniyor, depodan silinmiş belgeler indeksten düşülüyor. Güncellenen belgeyi yakalamak için her parçanın yanında belge içeriğinin SHA-256 özetini saklıyorum; özet uyuşmazsa belge yeniden indeksleniyor. Bu yaklaşımın bedeli, eşik aşıldıktan sonraki ilk aramanın indeksleme süresini ödemesi. Deneylerde bu süreyi ölçtüm: 20 belge için ilk arama 8 saniye, sonrakiler 0.9 saniye.

Güvenlik tarafında, Greshake ve arkadaşlarının tanımladığı dolaylı prompt injection saldırısına [9] karşı spotlighting uyguluyorum: retrieval ile gelen her parça DOC_START ve DOC_END işaretleri arasına alınıyor ve sistem talimatında bu sınırlar arasındaki içeriğin veri olduğu, talimat olmadığı açıkça söyleniyor. Ayrıca model, cevabın belgelerden çıkarılamadığı durumda bunu açıkça söylemeye yönlendiriliyor; canlı testte corpus dışı bir soruya modelin cevap uydurmak yerine cevap veremediğini söylediğini gözledim.

3.5 Yazılım Mimarisi Kararları

Çözümü katmanlı mimariyle kurdum: Core projesi soyutlamaları ve iş kurallarını, Persistence belge deposunu, Keyword, Embedding ve VectorRag projeleri üç arama katmanını, Api projesi de HTTP yüzeyini içeriyor. Üst katmanlar alt katmanları yalnızca arayüz üzerinden tanıyor; bağımlılıklar dependency injection ile veriliyor. Bu tercihin pratik karşılığını testlerde gördüm: OpenAI ve Qdrant yerine sahte uygulamalar takarak 37 testin çoğunu dış bağımlılık olmadan koşturuyorum.

- Result deseni: servisler hata durumunda exception fırlatmak yerine açıklayıcı bir sonuç nesnesi dönüyor. Controller bu nesneyi tek satırla HTTP cevabına çeviriyor; beklenmeyen hatalar da merkezi bir middleware'de yakalanıp ayrıntı sızdırmayan bir gövdeyle dönülüyor.
- Metrik kaydı: her aramanın hangi katmanda kaç milisaniyede cevaplandığı, decorator deseniyle pipeline'ı saran ayrı bir sınıfta tutuluyor. Router'ın tek sorumluluğu bozulmuyor.

- Yapılandırma ve sırlar: OpenAI anahtarı user-secrets üzerinden geliyor, kod tabanında hiçbir anahtar yok. Rate limiter IP başına dakikada 120 isteğe izin veriyor.
- Deney altyapısı: deneyler, API ile birebir aynı servis kayıtlarını kullanan ayrı bir konsol uygulamasında koşuyor. Ölçülen şey production pipeline'ının kendisi, kopyası değil.

4. Deneysel Çalışma ve Bulgular

4.1 Değerlendirme Yöntemi

Cevap kalitesini RAGAS çerçevesinin [7] iki metriğiyle ölçtüm: faithfulness ve answer relevancy. Puanlamayı GPT-4o-mini'ye hakem rolünde yaptırдым [8]; hakem her sorgu, cevap ve kaynak üçlüsüne 0 ile 1 arasında iki puan ve kısa bir gerekçe veriyor. Tutarlılık için hakemin sıcaklık değerini sıfıra sabitledim ve çıktığı yapılandırılmış JSON formatına zorladım. Retrieval başarısını ayrıca deterministik bir metrikle ölçtüm: hit@3, yani doğru belgenin ilk üç kaynak arasında olup olmaması. Deney corpus'u 15 el yazımı belge ve üzerine deterministik üretilen dolgu belgelerinden oluşuyor; aynı deney her koşuda aynı corpus'u görüyor.

4.2 Katman Geçiş Doğruluğu

İlk deney mimarinin temel iddiasını sınıyor: geçişler doğru eşikte mi gerçekleşiyor? Üç farklı eşik konfigürasyonunda, eşiklerin tam sınırındaki ve çevresindeki belge sayılarıyla toplam 30 ölçüm yaptım. Router 30 ölçümün 30'unda beklenen katmanı seçti. Sınırın bir altı ve bir üstü ayrı ayrı denendiği için kayma (off-by-one) hatası olmadığını da bu deney gösteriyor.

4.3 Strateji Karşılaştırması

İkinci deney, adaptif yaklaşımı iki baseline ile karşılaştırıyor: her sorguda embedding araması yapan kurulum ve her sorguda tam RAG çalıştıran kurulum. Corpus 1 belgeden 30 belgeye adım adım büyütüldü; her adımda üç altın sorgu soruldu ve cevaplar hakeme puanlatıldı.

Strateji	Relevancy	Faithfulness	Ortalama süre	hit@3
Adaptif (önerilen)	0.55	0.75	~1 ms	%100
Yalnız embedding	0.43	0.59	369 ms	%100

Yalnız RAG	0.75	0.75	1862 ms	%100
------------	------	------	---------	------

Üç bulgu öne çıkıyor. Birincisi, doğru belgeye erişim (hit@3) üç stratejide de belge corpus'a girer girmez sağlanıyor; yani sorun erişimde değil. İkincisi, az belgede BM25 embedding aramasından daha alakalı sonuç veriyor ve bunu üç kademe daha kısa sürede, sıfır API maliyetiyle yapıyor. Üçüncüsü, tam RAG en kaliteli cevabı veriyor ama her sorguya yaklaşık iki saniye gecikme ve LLM maliyeti ekliyor. Adaptif mimarinin değer önerisi bu tabloda görünüyor: pahalı katmanı, corpus onu hak edene kadar devreye sokmamak.

4.4 Eşik Hassasiyeti

Üçüncü deney, eşik değerlerinin yerini sorguluyor. Corpus 10, 25, 50 ve 100 belge boyutlarına getirildi; her boyutta üç katman router atlanarak doğrudan sorgulandı ve cevapları hakeme puanlatıldı.

Corpus	Layer 1 relevancy	Layer 2 relevancy	Layer 3 relevancy	Layer 3 süre
10 belge	0.77	0.53	1.00	1.7 sn
25 belge	0.60	0.50	1.00	3.3 sn
50 belge	0.50	0.60	1.00	3.5 sn
100 belge	0.50	0.57	1.00	4.8 sn

Tablo iki şeyi gösteriyor. Layer 1'in kalitesi corpus büyüdükçe düşüyor; kelime eşleşmesi, belge sayısı arttıkça gürültüye boğuluyor. Layer 2 ise 0.55 bandında sabit. İki eğri 50 belge civarında kesişiyor; üretim eşiğini 50 seçmem bu gözleme dayanıyor. Layer 3 her boyutta tavan kalitede; ona geçişin 200 belgeye ertelenmesi kalite kararı değil, maliyet ve gecikme kararı.

4.5 Katman Kalite Asimetrisi

Deneyler sırasında dikkat çekici bir asimetri yakaladım. Aynı tip sorguda Layer 2'nin cevabı hakemden faithfulness 1.0 ama relevancy 0.2 aldı; Layer 3'ün cevabı iki metrikte de 1.0 aldı. Sebep şu: Layer 2 cevap olarak en yakın belgelerin özetlerini art arda veriyor. Bu çıktı

kaynaklara tamamen sadık ama soruyla ilgisiz parçalar da içeriyor. Layer 3 ise LLM ile soruya odaklı bir cevap sentezliyor. Bu asimetri, üst katmana geçişin ne kazandırdığını tek örnekte özetliyor: mesele doğru belgeyi bulmak değil, doğru cevabı kurmak.

5. Sonuç ve Gelecek Çalışmalar

5.1 Sonuç

Bu projede, RAG sistemlerinin kuruluş anındaki cold start problemine yönelik üç katmanlı, adaptif ve iki yönlü geçişli bir mimari tasarladım; .NET 8 üzerinde uçtan uca çalışan bir prototip olarak geliştirdim ve ölçtüm. Sistem boş bilgi tabanı ile bile ayakta kalıyor, veri biriktikçe stratejisini kendisi değiştiriyor ve bu geçişleri ölçülebilir biçimde raporluyor.

Deneyisel bulgular tasarım kararlarını destekledi: geçiş doğruluğu yüzde yüz, küçük corpus'ta BM25 embedding aramasından daha alakalı ve kıyas kabul etmez ölçüde ucuz, Layer 1 ile Layer 2'nin kesişimi seçtiğim eşğin üzerine oturuyor. Çalışmanın literatüre katkısı, hibrit aramayı paralel değil sıralı kurgulaması ve cold start'ı bir retrieval stratejisi seçimi problemi olarak ele almasıdır.

5.2 Gelecek Çalışmalar

- Kalıcı depolama: belge deposu şu an bellek içi; yeniden başlatmada veri kayboluyor. Üretim senaryosu için kalıcı bir depo ve Qdrant ile kalıcı senkronizasyon gerekiyor.
- Türkçe için kök bulma: tokenizer şu an ek farkındalığı olmadan çalışıyor. Zemberek gibi bir morfolojik çözümleyici Layer 1 kalitesini özellikle Türkçe corpus'ta yükseltebilir.
- Cümle duyarlı parçalama: sabit boyutlu chunking yerine cümle sınırlarına saygılı bir bölücü, Layer 3 bağlam kalitesini artırabilir.
- Eşiklerin kendiliğinden ayarlanması: geçiş eşikleri şu an yapılandırmadan geliyor. Hakem skorlarını sürekli izleyip eşığı kendi ayarlayan bir mekanizma doğal bir devam çalışması.
- Arka planda indeksleme: Layer 3'e geçiş sonrası ilk aramanın ödediği indeksleme bedeli, eşige yaklaşırken arka planda hazırlık yaparak sıfırlanabilir.
- Kimlik doğrulama, yetkilendirme ve denetim kaydı gibi üretim sertleştirmeleri.

Kaynakça

- [1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., K ttler, H., Lewis, M., Yih, W., Rockt schel, T., Riedel, S. ve Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems 33 (NeurIPS 2020), s. 9459-9474.
- [2] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M. ve Wang, H. (2024). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv  n baskı, arXiv:2312.10997.
- [3] Asai, A., Wu, Z., Wang, Y., Sil, A. ve Hajishirzi, H. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. The Twelfth International Conference on Learning Representations (ICLR 2024).
- [4] Masood, A. (2024). Hybrid RAG: A Comprehensive Survey. arXiv  n baskı, arXiv:2410.12837.
- [5] Chen, H. ve Xu, H. (2025). ColdRAG: Retrieval-Augmented Generation ile  neri Sistemlerinde Cold-Start Probleminin Ele Alınması. arXiv  n baskı, arXiv:2505.20773.
- [6] Robertson, S. ve Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), s. 333-389.
- [7] Es, S., James, J., Espinosa-Anke, L. ve Schockaert, S. (2024). RAGAS: Automated Evaluation of Retrieval Augmented Generation. Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL 2024). arXiv:2309.15217.
- [8] Zheng, L., Chiang, W., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E. ve Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. Advances in Neural Information Processing Systems 36 (NeurIPS 2023).
- [9] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T. ve Fritz, M. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with

Indirect Prompt Injection. Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), s. 79-90.